

ProjectPilot AI

The Complete Supabase Database Guide

Step by step, from creating your account to a fully working, secure database — with every table, every setting, and every SQL command.

The ONLY database reference this team needs.
B.E. Computer Engineering — Mumbai University · Team of 4

0. What Is Supabase, In Simple Words

Supabase is a ready-made backend service built on top of **PostgreSQL** (a free, powerful database). Instead of you setting up a database server, a login system, file storage, and real-time updates all separately, Supabase gives you all four in one dashboard, for free at your project's scale.

Supabase Part	What It Replaces / Gives You
Database	A full PostgreSQL database — this is where every table in this document lives.
Auth	Ready-made login system — including "Sign in with GitHub," which we need anyway.
Storage	A place to upload files — we'll use this for meeting audio recordings.
Realtime	Instantly pushes updates to the frontend when data changes — perfect for live notifications, no extra setup needed.

Why this matters for an 8-month college project: it removes weeks of infrastructure work (building your own login system, file server, and live-update system) so your team can spend that time on the actual AI features.

1. Create Your Supabase Project

1. Go to **supabase.com** and click **Start your project**. Sign up using your GitHub account (recommended, since we'll connect GitHub later anyway).
2. Click **New Project**. Choose or create an Organization (just name it after your team, e.g. "TeamZenith").
3. Fill in the project form:
 - **Name:** `projectpilot-ai`
 - **Database Password:** generate a strong one and **save it somewhere safe** (a shared team password manager or private note) — you'll need it later and Supabase will not show it again
 - **Region:** pick the one closest to you (e.g. Mumbai/Singapore) for faster response times
 - **Pricing Plan:** Free tier is enough for the entire project
4. Click **Create new project** and wait 1–2 minutes while Supabase sets everything up.

Only one person should create the project. Once it exists, go to **Project Settings → Team** and invite the other 3 teammates by email so everyone has dashboard access.

Get your keys (you'll need these constantly)

Go to **Project Settings → API**. You'll see three important values — copy all three into a shared, private note:

Key	Safe to use where?	What it's for
<code>Project URL</code>	Frontend + Backend	The address of your Supabase project
<code>anon public</code> key	Frontend (safe)	Used by the website; Row Level Security (Section 6) keeps data safe even though this key is public
<code>service_role</code> key	Backend ONLY — never frontend	Bypasses all security rules; only your FastAPI server should ever use this

Critical rule: never put the `service_role` key in frontend code or commit it to GitHub. Anyone with that key has full access to your entire database. Keep it only in your backend's `.env` file, and make sure `.env` is in `.gitignore`.

Also go to **Project Settings → Database** and copy the **Connection String** (choose "Session pooler" for normal backend use) — your FastAPI backend will use this to talk to Postgres directly.

2. Turn On the Extra Tools We Need

Go to **Database → Extensions** in the Supabase dashboard and make sure these are turned on:

Extension	Why we need it
<code>pgcrypto</code>	Already on by default. Lets every table generate random unique IDs automatically.
<code>vector</code>	Turn this ON manually. Needed later for the AI Chatbot feature, to store "meaning" of text so the chatbot can search it (this is called embedding search).

To turn on `vector` : search for "vector" in the Extensions page and toggle it on. That's it — no SQL needed for this step.

3. Build the Schema — Step by Step

Go to the **SQL Editor** tab in Supabase (left sidebar). This is where your team will paste and run SQL commands to build every table. Run each block below **one at a time, in this exact order** (later tables reference earlier ones, so order matters). Click **Run** after pasting each block.

Step 3.1 — People and Teams

Supabase already has a built-in `auth.users` table that handles login. We create a `profiles` table linked to it, to store extra info like GitHub username.

```
create table public.profiles (  
  id uuid primary key references auth.users(id) on delete cascade,  
  full_name text,  
  email text,  
  role text default 'developer' check (role in ('developer','lead','manager')),  
  github_username text,  
  jira_account_id text,  
  created_at timestamptz default now()  
);  
  
create table public.teams (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  github_repo_url text,  
  jira_project_key text,  
  created_by uuid references public.profiles(id),  
  created_at timestamptz default now()  
);  
  
create table public.team_members (  
  id uuid primary key default gen_random_uuid(),  
  team_id uuid references public.teams(id) on delete cascade,  
  user_id uuid references public.profiles(id) on delete cascade,  
  role_in_team text default 'member' check (role_in_team in ('member','lead')),  
  joined_at timestamptz default now(),  
  unique(team_id, user_id)  
);
```

Why a separate `team_members` table instead of just a `team_id` on profiles? Because it lets us mark someone as a "lead" per team, which we use later to keep burnout scores private to leads only (Section 6).

Step 3.2 — Sprints and Tickets

```
create table public.sprints (  
  id uuid primary key default gen_random_uuid(),  
  team_id uuid references public.teams(id) on delete cascade,  
  name text not null,  
  start_date date,  
  end_date date,  
  planned_velocity numeric,  
  actual_velocity numeric,  
  status text default 'planned' check (status in ('planned','active','completed')),  
  created_at timestamptz default now()  
);  
  
create table public.tickets (  
  id uuid primary key default gen_random_uuid(),  
  team_id uuid references public.teams(id) on delete cascade,  
  sprint_id uuid references public.sprints(id) on delete set null,  
  jira_ticket_key text,  
  title text not null,  
  description text,  
  status text default 'todo' check (status in ('todo','in_progress','in_review','done')),  
  story_points numeric,  
  priority text default 'medium' check (priority in ('low','medium','high','critical')),  
  assignee_id uuid references public.profiles(id) on delete set null,  
  ai_generated boolean default false,  
  created_at timestamptz default now(),  
  updated_at timestamptz default now()  
);
```

Step 3.3 — GitHub Activity (Commits and Pull Requests)

```
create table public.commits (  
  id uuid primary key default gen_random_uuid(),  
  ticket_id uuid references public.tickets(id) on delete set null,  
  team_id uuid references public.teams(id) on delete cascade,
```

```

author_id uuid references public.profiles(id) on delete set null,
sha text not null unique,
message text,
files_changed jsonb,
lines_added integer default 0,
lines_removed integer default 0,
committed_at timestamptz not null,
created_at timestamptz default now()
);

create table public.pull_requests (
  id uuid primary key default gen_random_uuid(),
  ticket_id uuid references public.tickets(id) on delete set null,
  team_id uuid references public.teams(id) on delete cascade,
  author_id uuid references public.profiles(id) on delete set null,
  pr_number integer not null,
  title text,
  lines_added integer default 0,
  lines_removed integer default 0,
  files_changed integer default 0,
  test_coverage_delta numeric,
  review_count integer default 0,
  status text default 'open' check (status in ('open','merged','closed')),
  opened_at timestamptz,
  merged_at timestamptz,
  created_at timestamptz default now()
);

```

This table's `test_coverage_delta`, `lines_added/removed`, and `review_count` fields are exactly what feeds the Code-Aware Risk Detection feature.

Step 3.4 — Risk Scores (powers 3 features at once)

```

create table public.risk_scores (
  id uuid primary key default gen_random_uuid(),
  team_id uuid references public.teams(id) on delete cascade,
  risk_type text not null check (risk_type in ('delay','code_aware','burnout')),
  entity_type text not null check (entity_type in ('ticket','pull_request','user')),
  entity_id uuid not null,
  score numeric not null check (score >= 0 and score <= 100),
  shap_explanation jsonb,
  model_version text,
  computed_at timestamptz default now()
);

```

One table, three jobs: the `risk_type` column decides whether a row is a Delay & Risk Predictor score, a Code-Aware Risk score, or a Burnout score. The `shap_explanation` column (stored as JSON, e.g. `{"reasons": [...]}`) is what powers Explainable Risk Scores — it's shared by all three, not a separate feature.

Step 3.5 — Meetings and Action Items

```

create table public.meetings (
  id uuid primary key default gen_random_uuid(),
  team_id uuid references public.teams(id) on delete cascade,
  title text,
  audio_url text,
  transcript_text text,
  meeting_date timestamptz,
  uploaded_by uuid references public.profiles(id),
  created_at timestamptz default now()
);

create table public.action_items (
  id uuid primary key default gen_random_uuid(),
  meeting_id uuid references public.meetings(id) on delete cascade,
  ticket_id uuid references public.tickets(id) on delete set null,
  owner_id uuid references public.profiles(id),
  description text not null,
  due_date date,
  status text default 'pending' check (status in ('pending','verified_done','flagged_incomplete')),
  verified_at timestamptz,
  created_at timestamptz default now()
);

```

The `status` field is the whole Closed-Loop Meeting Verification feature: a background job flips it to `verified_done` or `flagged_incomplete` after checking GitHub/Jira on the due date.

Step 3.6 — Workload and What-If Simulation

```

create table public.workload_snapshots (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references public.profiles(id) on delete cascade,
  team_id uuid references public.teams(id) on delete cascade,
  week_start date not null,
  commits_count integer default 0,
  prs_count integer default 0,
  ticket_load integer default 0,
  after_hours_ratio numeric default 0,
  weekend_ratio numeric default 0,
  created_at timestamptz default now(),
  unique(user_id, week_start)
);

create table public.simulation_runs (
  id uuid primary key default gen_random_uuid(),
  sprint_id uuid references public.sprints(id) on delete cascade,
  team_id uuid references public.teams(id) on delete cascade,
  scenario_description text not null,
  input_params jsonb,
  output_results jsonb,
  run_by uuid references public.profiles(id),
  created_at timestamptz default now()
);

```

Step 3.7 — Reports, Notifications, Chat

```

create table public.reports (
  id uuid primary key default gen_random_uuid(),
  team_id uuid references public.teams(id) on delete cascade,
  week_start date not null,
  technical_version_text text,
  stakeholder_version_text text,
  generated_at timestamptz default now()
);

create table public.notifications (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references public.profiles(id) on delete cascade,
  team_id uuid references public.teams(id) on delete cascade,
  type text not null,
  message text not null,
  related_entity_type text,
  related_entity_id uuid,
  priority_score numeric default 0,
  is_read boolean default false,
  created_at timestamptz default now()
);

create table public.chat_messages (
  id uuid primary key default gen_random_uuid(),
  team_id uuid references public.teams(id) on delete cascade,
  user_id uuid references public.profiles(id) on delete cascade,
  role text not null check (role in ('user','assistant')),
  content text not null,
  referenced_entities jsonb,
  created_at timestamptz default now()
);

```

Step 3.8 — Chatbot Memory (for smart, grounded answers)

```

create table public.document_embeddings (
  id uuid primary key default gen_random_uuid(),
  team_id uuid references public.teams(id) on delete cascade,
  source_type text not null check (source_type in ('ticket','meeting','report')),
  source_id uuid not null,
  content_chunk text not null,
  embedding vector(384),
  created_at timestamptz default now()
);

```

This table lets the chatbot "search by meaning" instead of exact keywords. `vector(384)` matches the Sentence Transformers model size we planned to use — if your team picks a different embedding model later, just check its output size and match the number here.

Step 3.9 — Speed: Add Indexes

Indexes make lookups fast as your data grows. Run this after all tables above exist:

```

create index idx_tickets_team on public.tickets(team_id);

```

```

create index idx_tickets_sprint on public.tickets(sprint_id);
create index idx_tickets_assignee on public.tickets(assignee_id);
create index idx_commits_ticket on public.commits(ticket_id);
create index idx_pr_ticket on public.pull_requests(ticket_id);
create index idx_risk_entity on public.risk_scores(entity_type, entity_id);
create index idx_risk_team_type on public.risk_scores(team_id, risk_type);
create index idx_notifications_user_unread on public.notifications(user_id, is_read);
create index idx_workload_user_week on public.workload_snapshots(user_id, week_start);
create index idx_action_items_status on public.action_items(status, due_date);

```

Step 3.10 — Auto-Update Timestamps

This makes `updated_at` on tickets update itself automatically whenever a ticket changes, so nobody has to remember to set it manually:

```

create or replace function public.set_updated_at()
returns trigger as $$
begin
    new.updated_at = now();
    return new;
end;
$$ language plpgsql;

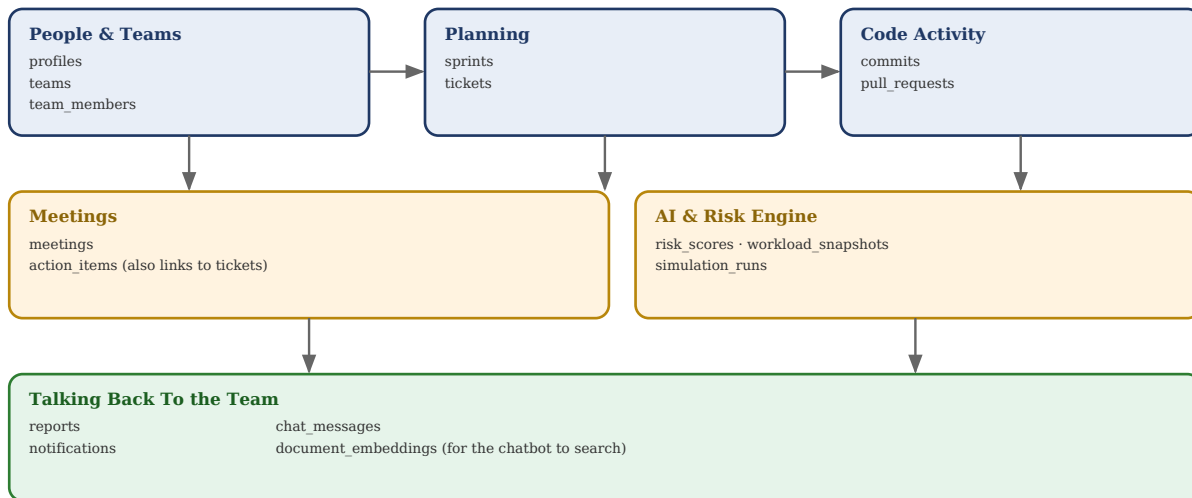
create trigger trg_tickets_updated_at
before update on public.tickets
for each row execute function public.set_updated_at();

```

Checkpoint: go to **Table Editor** in the sidebar. You should now see all 15 tables listed. If any are missing, scroll up in the SQL Editor to see which block returned an error, fix it, and re-run just that block.

4. Understand the Schema at a Glance

15 tables, grouped into 6 logical areas. Every arrow below is a real foreign key you just created in Section 3.



raw activity flows down into the AI engine → results flow into reports, alerts, and chat

5. Turn On Row Level Security (Very Important)

Row Level Security (RLS) means: even though your `anon` key is public (used by the whole website), every person can only see and edit the data they're actually allowed to. Without RLS turned on, **anyone with your project URL could read your entire database.**

Step 5.1 — Turn on RLS for every table

```
alter table public.profiles enable row level security;
alter table public.teams enable row level security;
alter table public.team_members enable row level security;
alter table public.sprints enable row level security;
alter table public.tickets enable row level security;
alter table public.commits enable row level security;
alter table public.pull_requests enable row level security;
alter table public.risk_scores enable row level security;
alter table public.meetings enable row level security;
alter table public.action_items enable row level security;
alter table public.workload_snapshots enable row level security;
alter table public.simulation_runs enable row level security;
alter table public.reports enable row level security;
alter table public.notifications enable row level security;
alter table public.chat_messages enable row level security;
alter table public.document_embeddings enable row level security;
```

Step 5.2 — Create two helper functions

These check "is this person on this team" and "is this person a lead on this team" — we'll reuse them in every policy below instead of repeating the same logic.

```
create or replace function public.is_team_member(check_team_id uuid)
returns boolean as $$
  select exists (
    select 1 from public.team_members
    where team_id = check_team_id and user_id = auth.uid()
  );
$$ language sql security definer;

create or replace function public.is_team_lead(check_team_id uuid)
returns boolean as $$
  select exists (
    select 1 from public.team_members
    where team_id = check_team_id and user_id = auth.uid() and role_in_team = 'lead'
  );
$$ language sql security definer;
```

Step 5.3 — Standard policy pattern (repeat for most tables)

Most tables follow the same rule: "you can see and edit it if you belong to the team it belongs to." Here it is for `tickets` — repeat this exact pattern (just swap the table name) for `sprints`, `commits`, `pull_requests`, `meetings`, `action_items`, `workload_snapshots`, `simulation_runs`, `reports`, `chat_messages`, and `document_embeddings`:

```
create policy "Team members can view tickets"
on public.tickets for select
using (public.is_team_member(team_id));

create policy "Team members can insert tickets"
on public.tickets for insert
with check (public.is_team_member(team_id));

create policy "Team members can update tickets"
on public.tickets for update
using (public.is_team_member(team_id));
```

Step 5.4 — The one exception: Burnout scores are private to leads

This is where the design decision from our earlier planning ("burnout signals are shown privately to the team lead only") actually gets enforced — not just as a UI rule, but as a real database rule nobody can bypass:

```
create policy "Non-burnout risk scores visible to team, burnout visible to leads only"
on public.risk_scores for select
using (
  (risk_type != 'burnout' and public.is_team_member(team_id))
  or
  public.is_team_lead(team_id)
);
```

```
(risk_type = 'burnout' and public.is_team_lead(team_id))
);
```

Step 5.5 — Teams, team_members, and profiles

```
create policy "Members can view their teams"
on public.teams for select
using (public.is_team_member(id));

create policy "Members can view their team roster"
on public.team_members for select
using (public.is_team_member(team_id));

create policy "Users can view teammate profiles"
on public.profiles for select
using (
  id = auth.uid()
  or exists (
    select 1 from public.team_members tm1
    join public.team_members tm2 on tm1.team_id = tm2.team_id
    where tm1.user_id = auth.uid() and tm2.user_id = public.profiles.id
  )
);

create policy "Users can update their own profile"
on public.profiles for update
using (id = auth.uid());
```

Step 5.6 — Notifications: strictly private

```
create policy "Users see only their own notifications"
on public.notifications for select
using (user_id = auth.uid());

create policy "Users can mark their own notifications read"
on public.notifications for update
using (user_id = auth.uid());
```

Before your demo day: test RLS by logging in as two different test users on two different teams, and confirming neither can see the other's tickets, reports, or notifications. This takes 10 minutes and prevents an embarrassing "everyone can see everyone's data" moment in front of your guide.

6. Set Up GitHub Login (Supabase Auth)

Supabase can handle "Sign in with GitHub" for you, out of the box — no need to build this yourself.

1. In GitHub: go to **Settings** → **Developer settings** → **OAuth Apps** → **New OAuth App**.
2. Fill in: Application name (`ProjectPilot AI`), Homepage URL (your dev URL, e.g. `http://localhost:3000`), and Authorization callback URL — get this exact URL from Supabase in the next step.
3. In Supabase: go to **Authentication** → **Providers** → **GitHub**, toggle it on. Supabase shows you the exact callback URL to paste into GitHub.
4. Copy the **Client ID** and **Client Secret** GitHub gives you, and paste them into the Supabase GitHub provider settings. Save.

In your Next.js frontend, logging in becomes a single line of code:

```
import { createClient } from '@supabase/supabase-js'
const supabase = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL, process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY)

// Trigger GitHub login from a button click:
await supabase.auth.signInWithOAuth({ provider: 'github' })
```

After login, insert a matching row into `public.profiles` using the new user's `id` (Supabase gives you a database trigger option for this — ask your backend lead to set up a simple "on new user → create profile" trigger, or do it manually in your first API call after login).

7. Set Up Storage (For Meeting Audio Files)

1. In Supabase, go to **Storage** in the sidebar, click **New bucket**.
2. Name it `meeting-audio`. Leave it **Private** (not public) — meeting recordings shouldn't be publicly accessible.
3. Add a storage policy so only team members can upload/download — go to the bucket's **Policies** tab and add one allowing authenticated users to upload and read files (start simple; refine later if needed).

When a user uploads a recording from the frontend, save the returned file path into the `meetings.audio_url` column so your backend knows where to find it for the Whisper transcription job.

8. Turn On Realtime (For Live Notifications)

Instead of building your own live-update system, let Supabase push updates straight to the frontend.

- 1 In Supabase, go to **Database → Replication**.
- 2 Find the `notifications` table and toggle it on for realtime.

Or via SQL:

```
alter publication supabase_realtime add table public.notifications;
```

Then in the frontend, subscribe to live changes:

```
supabase
  .channel('notifications')
  .on('postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'notifications', filter: 'user_id=eq.${userId}' },
    (payload) => { /* show the new notification instantly */ }
  )
  .subscribe()
```

This one setting replaces what would otherwise be a whole custom WebSocket server — a genuine time-saver for an 8-month timeline.

9. Connect Your Backend (FastAPI)

Your backend does the heavy lifting (ML models, AI agents, business logic), so it should talk to Postgres directly rather than only through the Supabase client.

Step 9.1 — Environment variables

Create a `.env` file in your backend (and add `.env` to `.gitignore` immediately):

```
SUPABASE_URL=https://your-project-ref.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key-here
DATABASE_URL=postgresql://postgres:[your-db-password]@[connection-string-from-section-1]
```

Step 9.2 — Connect with SQLAlchemy

```
pip install sqlalchemy asyncpg python-dotenv
```

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker
import os

DATABASE_URL = os.getenv("DATABASE_URL").replace("postgresql://", "postgresql+asyncpg://")
engine = create_async_engine(DATABASE_URL, echo=False)
AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)
```

Because the backend uses the `service_role` connection, it bypasses RLS entirely — which is correct, since your backend is trusted code that enforces its own permission checks in Python before touching the database.

10. Connect Your Frontend (Next.js)

```
npm install @supabase/supabase-js
```

Create a file `lib/supabase.js`:

```
import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)
```

Add to `.env.local` (these two are safe to expose in frontend code — RLS is what actually protects your data, not secrecy of this key):

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key-here
```

Now any page can fetch data respecting RLS automatically, e.g.:

```
const { data: tickets } = await supabase
  .from('tickets')
  .select('*')
  .eq('sprint_id', currentSprintId)
```

11. Test Everything Works — Final Checklist

- ☐ All 15 tables appear in **Table Editor**
- ☐ Ran every RLS policy in Section 5, and the Table Editor's "RLS enabled" badge shows green for all 16 tables (15 + note there are 16 total including document_embeddings)
- ☐ Logged in successfully via "Sign in with GitHub" and a matching row appeared in `profiles`
- ☐ Created one test team, added yourself to `team_members`, and confirmed you can see it from the frontend
- ☐ Inserted one test ticket via SQL Editor and confirmed it shows up through the frontend Supabase client
- ☐ Uploaded a test audio file to the `meeting-audio` bucket and confirmed you can retrieve its URL
- ☐ Inserted a test row into `notifications` and confirmed the frontend receives it instantly via Realtime
- ☐ Logged in as a second test user on a *different* team and confirmed they cannot see the first team's data anywhere
- ☐ Confirmed `.env` and `.env.local` are both in `.gitignore` and were never committed to GitHub

Common mistakes to avoid

Mistake	Why it hurts you
Forgetting to enable RLS on a table	That table becomes fully public to anyone with your project URL — a real security hole, not just a bug
Committing the <code>service_role</code> key to GitHub	Anyone who finds it has full read/write access to your entire database
Using <code>timestamp</code> instead of <code>timestampz</code>	You'll get confusing time-zone bugs later — always use <code>timestampz</code> as shown above
Skipping indexes until "later"	Queries feel fast with 10 test rows and become painfully slow with real data by month 5
Not testing RLS with two different accounts	The single most common demo-day embarrassment for student projects using Supabase

This document is the complete, final database reference for ProjectPilot AI. Every table, every security rule, and every connection step your team needs lives here — refer back to it any time the database comes up.